

Introducción

El presente trabajo modela los requisitos de un portal web que centraliza la oferta de empresas de transporte de carga y mudanzas en Colombia. A partir del enunciado se identifican los actores, las funciones principales del sistema, las entidades del dominio y las reglas de negocio necesarias para estructurar la solución. Se emplean vistas UML de comportamiento y estructura, junto con un modelo entidad-relación para el diseño de datos.

El propósito de los modelos es representar de manera consistente cómo interactúan los usuarios con el sistema, cómo se relacionan los objetos principales de la aplicación y cómo puede almacenarse la información sin perder reglas como los precios por población, los recargos por peso, el historial laboral de los empleados y la trazabilidad de las solicitudes.

1. Modelado de actores y casos de uso

El diagrama de casos de uso representa la vista dinámica o de comportamiento del sistema. Los actores se definen según las responsabilidades descritas en el caso.

Actor	Responsabilidad principal
Cliente	Consulta empresas y servicios, crea solicitudes, consulta su estado, acepta o rechaza propuestas y realiza el pago anticipado.
Empresa de mudanzas	Administra servicios, precios, poblaciones, vehículos, empleados y solicitudes; además registra la ejecución de los servicios.
Empleado	Consulta asignaciones y participa en los servicios ejecutados. Puede registrar información operativa si la empresa le concede acceso.
Administrador del portal	Mantiene el catálogo general de servicios, poblaciones y datos maestros del portal.

Los casos de uso principales son: consultar empresas y servicios; crear una solicitud; consultar su estado; aceptar o rechazar una propuesta; registrar el pago; gestionar la oferta de servicios, poblaciones y precios; administrar vehículos y empleados; resolver solicitudes; registrar la ejecución real del servicio; consultar asignaciones y gestionar el catálogo general.

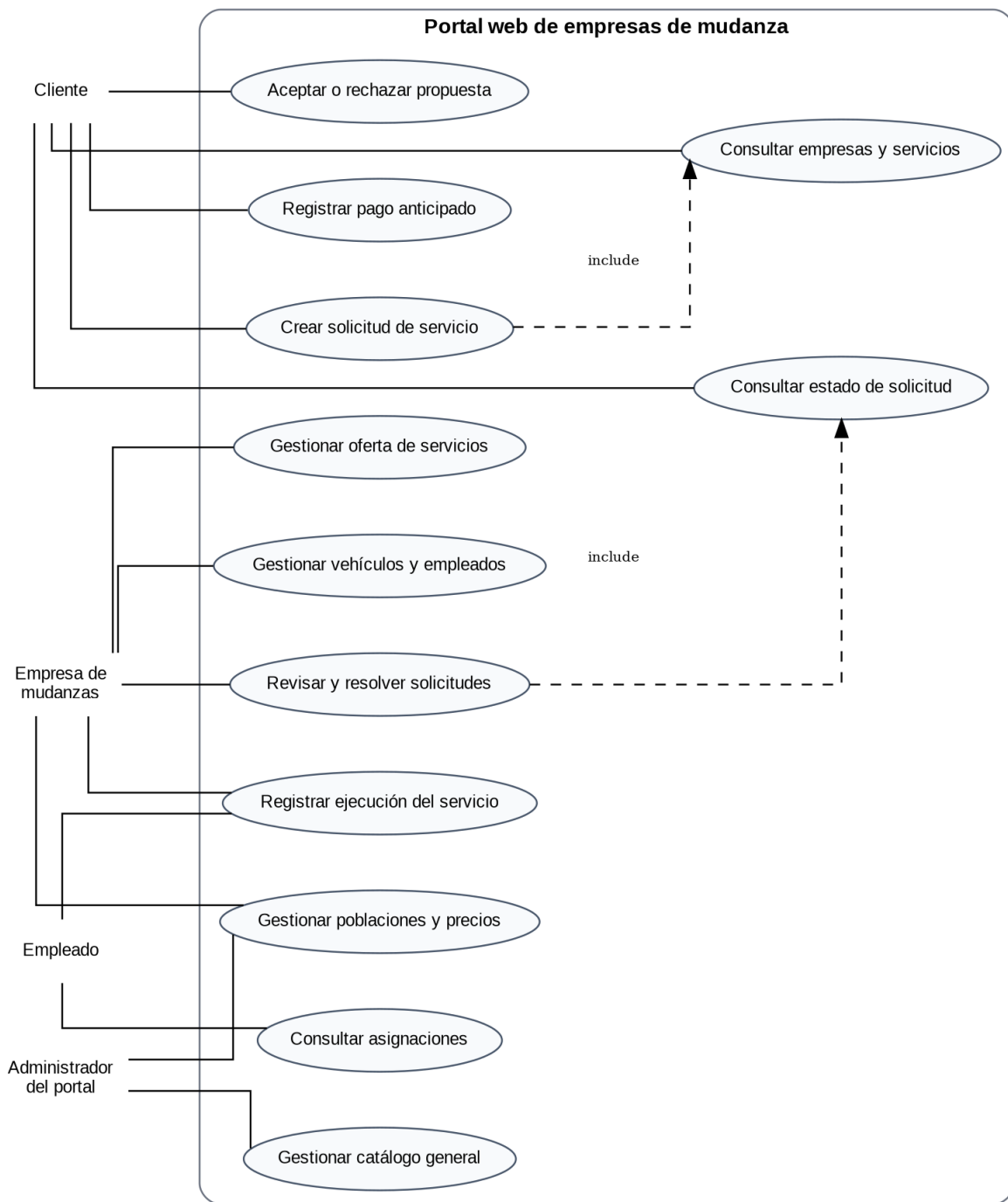


Figura 1. Diagrama UML de casos de uso del portal de mudanzas.

Descripción de los casos de uso más relevantes

Caso de uso	Actor	Entrada principal	Resultado
Crear solicitud	Cliente	Empresa, servicios, origen, destino y fecha	Solicitud identificada con código único.
Resolver solicitud	Empresa	Solicitud recibida, disponibilidad y precios	Solicitud aceptada o rechazada, con fecha de resolución.
Registrar ejecución	Empresa / Empleado	Fecha real, duración y empleados participantes	Trazabilidad de los servicios realizados.
Gestionar oferta	Empresa	Servicio, población y precio/hora	Servicio disponible en una población con tarifa específica.
Gestionar recargos	Empresa	Umbral de kg y porcentaje	Regla de incremento para transporte pesado.

2. Diagrama de clases

El diagrama de clases representa la vista estática del sistema. Se modelan clases de negocio y clases asociativas necesarias para resolver relaciones de muchos a muchos y para almacenar atributos propios de esas relaciones.

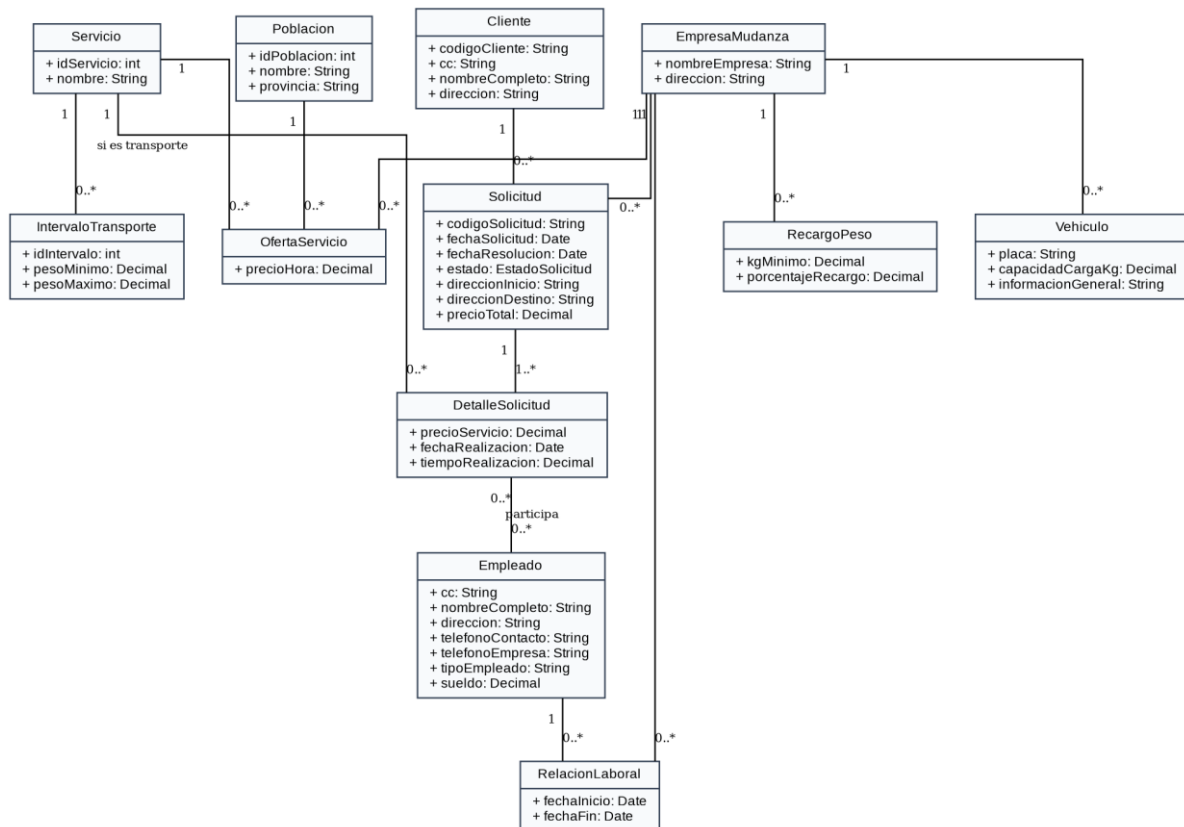


Figura 2. Diagrama de clases UML propuesto.

La clase OfertaServicio es fundamental porque el precio no depende únicamente del servicio o de la empresa: una misma empresa puede ofrecer un servicio en varias poblaciones con valores diferentes. Por ello, OfertaServicio vincula EmpresaMudanza, Servicio y Poblacion, almacenando el precio por hora.

Solicitud vincula al cliente con una empresa de mudanzas y registra el estado general del requerimiento. DetalleSolicitud representa cada servicio pedido dentro de la solicitud y permite conservar el precio acordado, la fecha real de ejecución y el tiempo empleado. De esta forma, los cambios futuros en las tarifas no modifican el historial de una solicitud ya resuelta.

RelacionLaboral permite conservar el historial de un empleado en una o varias empresas, incluyendo fechas de inicio y fin. La relación entre DetalleSolicitud y Empleado es de muchos a muchos porque varios empleados pueden participar en un servicio y un empleado puede intervenir en numerosas solicitudes.

Reglas de negocio representadas en el modelo de clases

- El nombre de cada empresa es único dentro del portal.
- Un servicio pertenece a un catálogo común para todas las empresas.
- El precio/hora se define por la combinación empresa-servicio-población.
- Los recargos por peso se definen por empresa y no por población.
- Los intervalos de peso para transporte son estándares del sector y se asocian al servicio de transporte.
- La especialidad de un empleado no restringe técnicamente los trabajos que puede realizar.
- Una solicitud aceptada puede registrar varios servicios ejecutados y varios empleados participantes.

3. Diseño de datos mediante diagrama E/R

El modelo entidad-relación traduce la estructura del dominio a un diseño orientado a base de datos. Se definen claves primarias, claves foráneas y entidades asociativas para conservar correctamente las cardinalidades.

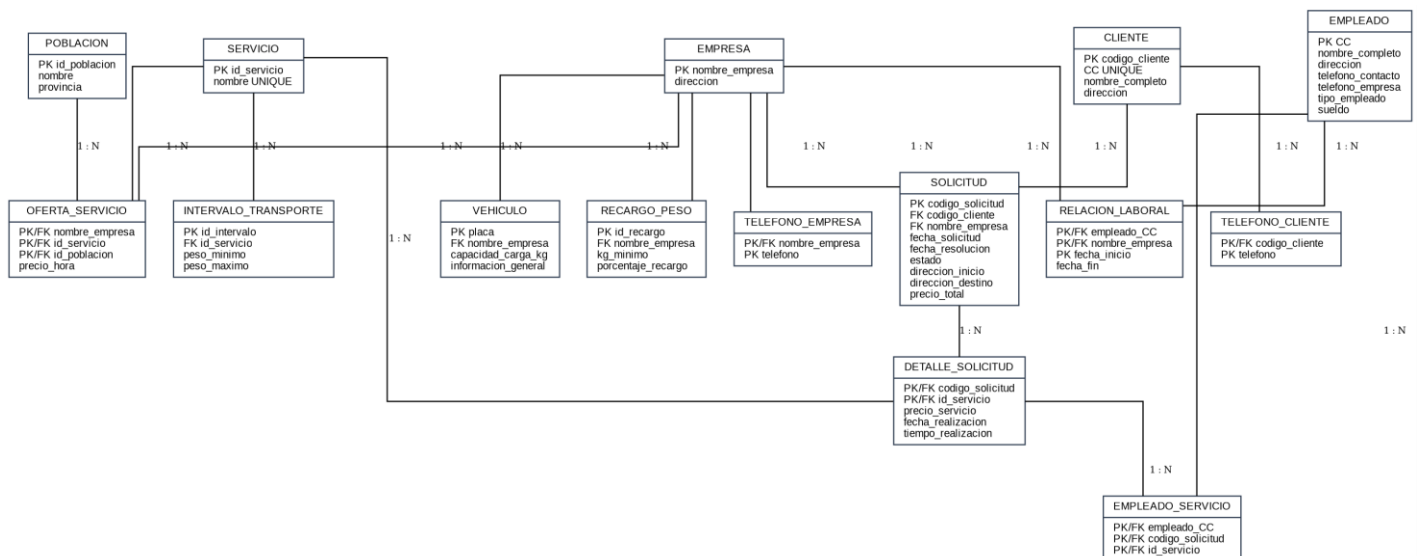


Figura 3. Diagrama E/R lógico para la base de datos.

Supuestos semánticos complementarios

1. Cada empresa se identifica por un nombre único. Si en una implementación real se desea permitir cambios de nombre, se recomienda añadir un identificador interno numérico y mantener el nombre con restricción UNIQUE.
2. Los teléfonos se modelan en tablas independientes porque una empresa o un cliente puede registrar varios números de contacto.
3. Una empresa puede prestar servicios en múltiples poblaciones y una población puede ser atendida por múltiples empresas.
4. Cada combinación empresa-servicio-población tiene como máximo una tarifa vigente dentro del modelo básico. Si se requiere historial de tarifas, se añadirían fecha_inicio y fecha_fin a OFERTA_SERVICIO.
5. Las direcciones de origen y destino deben pertenecer a poblaciones en las que la empresa ofrezca el servicio solicitado. El destino puede ser nulo cuando el servicio no implique desplazamiento.
6. El precio de cada servicio solicitado se copia a DETALLE_SOLICITUD al momento de resolver la solicitud, evitando que modificaciones posteriores de tarifas alteren los datos históricos.
7. Los recargos por peso se aplican únicamente a servicios de transporte y se almacenan por empresa. Los umbrales pueden ser múltiples, por ejemplo 500 kg +10 % y 750 kg +15 %.
8. Los intervalos de peso regulados por ley se consideran datos maestros del servicio de transporte y no dependen de una empresa concreta.
9. Un empleado puede trabajar para diferentes empresas en momentos distintos. RELACION_LABORAL conserva las fechas de cada vínculo.
10. El campo tipo_empleado describe la especialidad, pero no impone una restricción sobre las tareas que puede ejecutar.
11. Los pagos se realizan antes de la prestación del servicio. El modelo puede registrar el estado o comprobante del pago si el alcance del proyecto se amplía, pero no requiere cuentas por cobrar posteriores.

4. Análisis para la selección de framework

Para cumplir el criterio de selección tecnológica se comparan tres alternativas de desarrollo web a partir de cinco criterios: facilidad de desarrollo, seguridad, acceso a datos, escalabilidad y mantenibilidad. La comparación se plantea de forma cualitativa porque el caso todavía no define restricciones de lenguaje, proveedor o infraestructura.

Criterio	Django	Laravel	Spring Boot
Facilidad de desarrollo	Alta: estructura integrada y administración automática.	Alta: sintaxis clara y amplio ecosistema web.	Media: mayor configuración y curva de aprendizaje.
Seguridad	Alta: mecanismos integrados de autenticación y protección web.	Alta: herramientas integradas para autenticación y validación.	Muy alta: ecosistema robusto para aplicaciones empresariales.
Persistencia de datos	Alta: ORM integrado y migraciones.	Alta: Eloquent ORM y migraciones.	Muy alta: JPA/Hibernate y amplio soporte empresarial.
Escalabilidad	Alta para el alcance propuesto.	Alta para aplicaciones web de tamaño medio y grande.	Muy alta, especialmente en arquitecturas empresariales distribuidas.
Mantenibilidad	Alta: convención y estructura	Alta: organización MVC y	Alta: arquitectura modular y

	clara.	comunidad amplia.	tipado estático.
--	--------	-------------------	------------------

Framework seleccionado

Para este caso se propone Django. La razón principal es que el portal requiere una base de datos relacional con numerosas entidades, autenticación de usuarios, administración de catálogos y formularios de gestión. Django facilita estas necesidades mediante su ORM, sistema de autenticación, validaciones, migraciones y panel administrativo. Además, permite desarrollar con rapidez un producto inicial sin impedir una evolución posterior hacia una arquitectura más escalable.

La selección no significa que Laravel o Spring Boot sean alternativas inadecuadas. Spring Boot sería especialmente conveniente si la organización ya trabaja con Java y requiere integraciones empresariales complejas; Laravel sería una opción sólida si el equipo tiene mayor experiencia con PHP. La decisión definitiva debería considerar también competencias del equipo, costos operativos y tecnología existente en la empresa de hosting.

Conclusiones

El modelado realizado permite representar el portal desde tres perspectivas complementarias. El diagrama de casos de uso describe cómo interactúan los actores con la aplicación; el diagrama de clases organiza las responsabilidades y relaciones de los objetos del dominio; y el modelo E/R define una estructura de datos capaz de conservar la información operativa e histórica requerida por el negocio.

Las entidades asociativas OfertaServicio, DetalleSolicitud y RelacionLaboral son esenciales para modelar correctamente las reglas del caso. Sin ellas se perdería información sobre tarifas por población, servicios concretos incluidos en una solicitud y periodos de vinculación de los empleados con las empresas.

Finalmente, la comparación de frameworks se realizó con cinco criterios, superando el mínimo de cuatro exigido en la rúbrica. Django se propone como alternativa adecuada para un primer desarrollo por su productividad, seguridad integrada y facilidad para trabajar con un modelo relacional de datos.